Plan de diseño: defensa por oleadas programando en Java

Oct 2, 2026 · @Yeray

El bucle central

Cada oleada dura unos 45 segundos y corre sola: tu trabajo es leer qué viene, cambiar el código y decidir si avanzas o repites.

1. **Parte de oleada.** Ves qué enemigos vienen y cuántos, antes de que salgan.
2. **Editar y comprar.** Cambias el script de cada torreta y gastas el dinero en sintaxis o en hardware.
3. **Desplegar.** La oleada corre sin que puedas tocar el código. Solo quedan clics de emergencia, como recargar a mano.
4. **Informe.** Dinero ganado, enemigos que se colaron y excepciones que saltaron, cada una con su línea.
5. **Avanzar o repetir.** Con el avance automático apagado, la misma oleada se repite y sigues cobrando.

El gancho es que el código sustituye a tus clics. En la oleada 1 recargas a mano y no das abasto; cada compra te quita un trabajo manual.

Los enemigos recorren un camino hasta el núcleo, que tiene 20 vidas por oleada. Cada enemigo que llega quita una. Las torretas van en huecos fijos junto al camino: empiezas con uno y compras hasta seis.

Qué programa el jugador

Cada torreta tiene su propio script, que se ejecuta entero 2 veces por segundo (un ciclo). El jugador solo escribe el cuerpo; la clase que lo envuelve está oculta hasta que compra clases.

Llamada	Qué hace	Disponible
`disparar()`	Dispara una bala al objetivo actual. Por defecto, el enemigo más cercano	Inicio
`recargar()`	Llena el cargador. La torreta no hace nada durante 3 ciclos	Inicio

Llamada	Qué hace	Disponible
`municion()`, `calor()`, `hayEnemigos()`	Lecturas de la propia torreta	Con `if`
`enemigoMasCercano()`, `apuntar(e)`	Elegir objetivo a mano	Con variables
`e.vida()`, `e.velocidad()`, `e.blindado()`, `e.distanciaAlNucleo()`	Datos de un enemigo	Con variables
`usarMunicion(tipo)`	Cambia entre `NORMAL` y `PERFORANTE`	Oleada 6
`enemigos()`	Lista de todos los enemigos en rango	Con `for`
`desatascar()`	Arregla un atasco	Con `try/catch`

Tres límites obligan a escribir buen código:

- **Cargador:** 12 balas. Disparar sin balas lanza una excepción.
- **Calor:** cada disparo suma 20 y cada ciclo disipa 30, con máximo 100. Un disparo por ciclo es sostenible; las ráfagas hay que medirlas. Sustituye a la energía que planteamos antes: es lo mismo, pero se ve en una barra.
- **Memoria:** 12 líneas por script. Es lo que más adelante hace valiosos los métodos.

Así crece el script de una torreta. En la oleada 1, gratis:

```
disparar();
```

En la oleada 3, con `if`, ya no recargas a mano:

```
if (municion() == 0) {
    recargar();
} else {
    disparar();
}
```

En la oleada 12, con `for`, eliges tú a quién disparar:

```
Enemigo objetivo = null;
for (Enemigo e : enemigos()) {
    if (objetivo == null || e.distanciaAlNucleo() <
objetivo.distanciaAlNucleo()) {
        objetivo = e;
    }
}
if (objetivo != null) {
    apuntar(objetivo);
    disparar();
}
```

Árbol de sintaxis

La tienda vende ocho piezas de Java en orden fijo, y cada una elimina un problema que el jugador ya está sufriendo.

Desbloqueo	Precio	Requiere	Llega hacia la oleada	Qué te quita de encima
`if / else` y comparaciones	25	Nada	2–3	Recargar a mano y disparar al aire
Variables	60	`if`	4–5	Cambiar de objetivo sin querer
`&&`, `||`, `!`	100	Variables	6–7	Anidar un `if` dentro de otro
`while`	200	`&&`	8–9	Copiar `disparar();` tres veces para hacer ráfagas
`for` y listas	450	`while`	11–13	Depender de `enemigoMasCercano()`
Métodos propios	900	`for`	15–16	Mantener el mismo código en cuatro torretas
`try / catch`	1.800	Métodos	18–20	Torretas colgadas por fallos que no puedes prever
Clases y herencia	4.000	`try/catch`	23–26	Que todas las torretas sean iguales

Reglas de la tienda:

- Cada compra abre una tarjeta con un ejemplo de cinco líneas que funciona tal cual en el juego.
- Usar sintaxis sin comprar no compila, y el mensaje lo dice claro: «Aún no has desbloqueado `while`».
- Todo el árbol se ve desde el principio con su precio, para que siempre haya un objetivo de ahorro.
- Los métodos se escriben en una biblioteca compartida por todas las torretas y no cuentan para el límite de 12 líneas.

Regla de diseño: cada pieza de sintaxis, bien usada, debe rendir al menos el doble que gastar ese dinero en hardware. Si no, el jugador comprará daño y no aprenderá nada.

Enemigos

Cada enemigo nuevo aparece una o dos oleadas antes de que puedas pagar la sintaxis que lo resuelve: primero sufres el problema, luego compras la solución.

Enemigo	Desde la oleada	Qué hace	Qué te obliga a usar
Básico	1	Camina hacia el núcleo	`if` para recargar solo
Regenerador	4	Se cura si pasa 1 segundo sin recibir daño	Variables, para no soltar al objetivo
Blindado	6	Recibe un 80 % menos de daño con munición normal	`&&`, para gastar perforante solo cuando compensa
Enjambre	8	Grupos de 10 enemigos débiles y juntos	`while`, para disparar ráfagas sin quemar la torreta
Corredor	11	Va tres veces más rápido y se cuela mientras disparas a otro	`for`, para priorizar al que está más cerca del núcleo
Mutante	14	Su punto débil cambia cada oleada	Métodos, para cambiar una línea y no cuatro scripts
Saboteador	17	Lanza pulsos que hacen fallar los sensores y atascan armas	`try/catch`

La munición perforante atraviesa el blindaje pero suma 40 de calor en vez de 20. Usarla contra todo quema la torreta.

Jefes. Cada 10 oleadas sale uno con 20 veces la vida de un enemigo normal de esa oleada. Si llega al núcleo, pierdes la oleada entera. El primer derribo paga de golpe el equivalente a tres oleadas. El jefe de la 10 alterna blindaje cada 5 segundos, así que examina justo lo aprendido hasta ahí.

Modificadores. A partir de la oleada 21, cada oleada añade uno a los enemigos base, y desde la 31, dos o tres combinados. Son los que impiden que el juego quede resuelto:

- **Sigiloso:** solo aparece en `enemigos()` a media distancia.
- **Explosivo:** al morir suma calor a las torretas cercanas; conviene matarlo lejos.
- **Divisor:** al morir se parte en dos más pequeños.
- **Rápido, blindado y regenerador** aplicados a cualquier tipo.

Excepciones

Una excepción sin capturar cuelga la torreta 3 segundos y muestra encima el error con su línea. Las previsibles se evitan con un `if`; las imprevisibles solo se arreglan con `try/catch`.

Excepción	Cuándo salta	Tipo	Cómo se resuelve
`SinMunicionException`	`disparar()` con el cargador vacío	Previsible	`if (municion() == 0) recargar();`
`SobrecalentamientoException`	`disparar()` con el calor a 100	Previsible	`if (calor() < 80) disparar();`
`NullPointerException`	Usar un enemigo que no existe o ya ha muerto	Previsible	`if (objetivo != null)`
`BucleInfinitoException`	Un `while` pasa de 1.000 vueltas en un ciclo	Previsible	Corregir la condición del bucle
`AtascoException`	Al azar al disparar con el calor por encima de 50	Imprevisible	`catch` y `desatascar()`
`InterferenciaException`	Leer un sensor durante un pulso del Saboteador	Imprevisible	`catch` y disparar al último objetivo conocido

Tres decisiones detrás de la tabla:

- `NullPointerException` es la de Java de verdad. Es el error que más van a ver fuera del juego, así que conviene que lo conozcan aquí.
- `BucleInfinitoException` no existe en Java. Hace falta para que un `while (true)` no congele el juego, y de paso enseña a revisar condiciones.
- Los atascos empiezan en la oleada 15, antes de poder comprar `try/catch`. Durante unas oleadas solo se evitan disparando en frío, a costa de cadencia, y eso crea las ganas de comprarlo.

El informe de cada oleada lista las excepciones por torreta y cuántos segundos estuvo colgada cada una. Es la pista principal para saber qué arreglar.

Economía y farmeo

La vida de los enemigos sube un 18 % por oleada y tu daño solo sube cuando compras algo o mejoras el código. Ese desfase es el que crea los muros.

```
\begin{aligned}
\text{vida}(n) &= 10 \cdot 1{,}18^{\,n-1} \\
\text{dinero por enemigo}(n) &= 2 \cdot 1{,}15^{\,n-1} \\
\text{enemigos}(n) &= \min(6 + 2n,\ 60)
\end{aligned}
```

Oleada	Enemigos	Vida por enemigo	Dinero por enemigo	Dinero por oleada
1	8	10	2	16
5	16	19	3,5	56
10	26	44	7	183
20	46	232	28	1.309
30	60	1.215	115	6.909

Una bala hace 5 de daño al empezar. Todos estos números son valores de partida para ajustar jugando, no cifras probadas.

Hardware

El hardware no tiene tope, pero cada nivel cuesta bastante más que el anterior y da lo mismo. Así sirve para pasar un muro, no para ganar el juego.

Mejora	Efecto por nivel	Primer nivel	Cada nivel siguiente
Daño	+25 %	20	×1,6
Cargador	+4 balas	15	×1,5
Memoria	+4 líneas por script	30	×1,8
Disipador	+5 de calor disipado por ciclo	40	×1,6
Procesador	+0,5 ciclos por segundo, máximo 5	50	×2
Torreta extra	Un hueco más, máximo 6	75	×3

Farmeo

Farmear es dejar que el código que ya tienes repita una oleada mientras tú escribes el siguiente.

- **Avance automático.** Un interruptor. Apagado, la oleada actual se repite sin parar y paga lo mismo cada vez.
- **Oleada perfecta.** Si no se cuela nadie, paga un 25 % más.
- **Velocidad ×3** en cualquier oleada que ya hayas superado sin fugas.
- **Edición en caliente.** Puedes editar mientras corre; el código nuevo entra en la siguiente pasada.
- **Contratos.** Tres retos opcionales a la vista, como «supera la oleada 9 con menos de 60 balas» o «sin ninguna excepción». Cada uno paga lo que dos o tres oleadas.

Un muro típico: te atascas en la oleada 12 y repites la 11, que paga 227, o 283 si es perfecta. `for` cuesta 450, así que son dos pasadas, menos de un minuto a velocidad ×3.

Regla de calibrado: ningún muro debe pedir más de 5 pasadas. Si un jugador de prueba necesita más, se baja el precio o se sube la recompensa.

Los contratos son lo que evita el aburrimiento. Dan una razón para reescribir el código durante el farmeo en vez de mirar la pantalla, y premian justo lo que quieres enseñar: gastar menos balas, no fallar, escribir más corto.

Fases de la partida

La partida pasa por cinco fases, y en cada una cambia lo que significa jugar bien.

Fase	Oleadas	Compras clave	Enemigos nuevos	Qué cambia para el jugador
A mano	1–3	`if / else`	Básico	Recargas con clics hasta que el primer `if` lo hace por ti
Un script	4–10	Variables, `&&`, `while`, segunda torreta	Regenerador, Blindado, Enjambre, Jefe 1	Scripts de 5 a 10 líneas. El jefe es el primer muro serio
Varias torretas	11–20	`for`, métodos, `try/catch`	Corredor, Mutante, Saboteador, Jefe 2	Tres o cuatro torretas. El problema pasa a ser el código duplicado y los fallos al azar
Tus clases	21–30	Clases y herencia	Un modificador por oleada, Jefe 3	Diseñas un tipo de torreta distinto para cada hueco
Infinito	31 en adelante	Nueva versión	Dos o tres modificadores combinados	Reescribes para cada combinación y los contratos se endurecen

Ritmo objetivo: una compra de sintaxis cada dos o tres oleadas hasta la 10, y cada tres a cinco después. Entre una y otra, el dinero va a hardware.

Duración objetivo, por comprobar con jugadores: Jefe 1 en 20 a 30 minutos y Jefe 2 en unas dos horas. Un principiante tardará más, porque el tiempo se le va en escribir, no en esperar.

Derrota y reinicio

Perder una oleada no borra nada: vuelves a la anterior con el avance automático apagado, es decir, a farmear.

- Cobras los enemigos que hayas matado antes de caer.
- El informe de derrota dice por qué punto del camino se colaron, qué torreta estuvo colgada y cuántas balas se gastaron en enemigos ya muertos.
- Conservas dinero, sintaxis y hardware.

Esto cambia lo que te propuse antes, que era reiniciar dinero y oleada al perder. Con farmeo en bucle sobra: el castigo ya es quedarte atascado, y mandar a un principiante a la oleada 1 lo echa del juego.

Nueva versión. Desde la oleada 30 puedes reiniciar por decisión propia. Vuelves a la oleada 1 con toda la sintaxis, un 50 % más de dinero para siempre y un mapa nuevo con dos caminos. Abre el nivel avanzado de la tienda: interfaces, excepciones propias con `throw` y lambdas. Queda fuera de la primera versión.

Riesgos y primera versión

La primera versión debería acabar en el Jefe 1: oleadas 1 a 10, cuatro piezas de sintaxis y cuatro enemigos. Todo lo demás espera a saber si eso engancha.

Qué entra:

- Oleadas 1 a 10 con Básico, Regenerador, Blindado, Enjambre y el jefe.
- `if`, variables, `&&` y `while`.
- Dos torretas y tres mejoras de hardware: daño, cargador y disipador.
- Cuatro excepciones: `SinMunicionException`, `SobrecalentamientoException`, `NullPointerException` y `BucleInfinitoException`.
- Avance automático, oleada perfecta, parte de oleada e informe. Los contratos se añaden después.

Riesgo	Por qué importa	Qué hacer
Balance sin probar	Las fórmulas y los precios están puestos a ojo	Un simulador sin gráficos que juegue las 10 oleadas con scripts de referencia y cuente pasadas por muro
Los primeros cinco minutos	Con una línea de código casi no hay decisiones	`if` al alcance en la oleada 2 y una recarga manual que agobie lo justo
Comprar daño en vez de sintaxis	El jugador avanza sin aprender	Comprobar en el simulador la regla del doble para cada pieza
Código del jugador que rompe el juego	Un `while (true)` o un `System.exit(0)` lo tumba todo	Límite de vueltas, lista cerrada de clases permitidas y ejecución en un hilo aparte

Riesgo	Por qué importa	Qué hacer
Farmeo pasivo	Si el jugador solo espera, se va	Regla de las 5 pasadas y contratos en cuanto la base funcione
Alcance	Clases y herencia es casi otro juego entero	No empezarla hasta que la primera versión enganche

Criterio para seguir: que varios compañeros de clase lleguen al Jefe 1 sin ayuda y por gusto.

Pregunta abierta: el tema. Que los enemigos sean bugs atacando un servidor encaja con las excepciones, pero está sin decidir.